

Работа с файлами

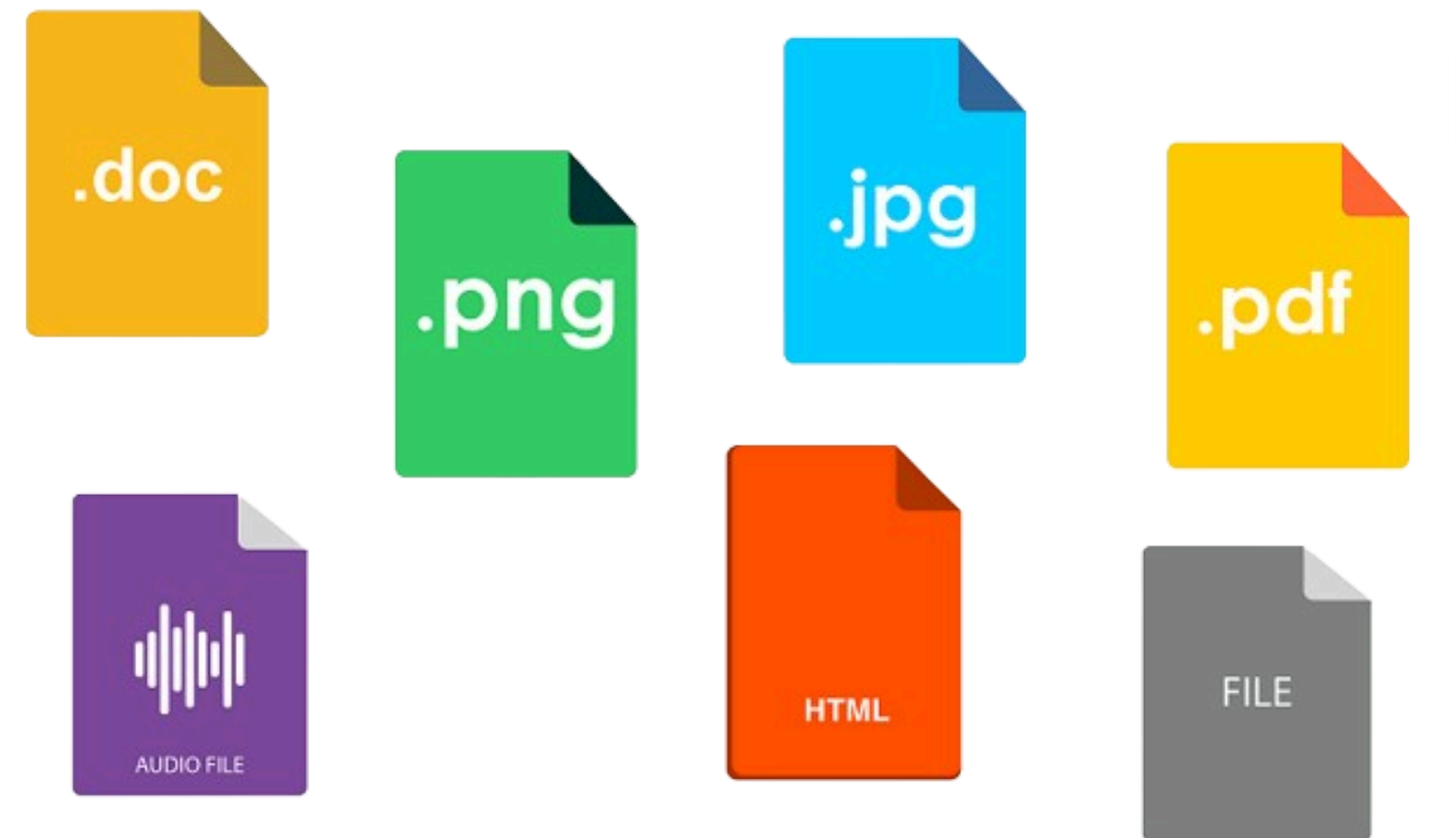




Что такое файл?

Файл - это контейнер в котором хранится определенный набор байт который может представлять из себя различную информацию: текст, изображение, аудио, видео, набор данных для программы или же сама программа.

Все что находится на вашем компьютере состоит из файлов, а жесткий диск или твердотельный накопитель (SSD) это хранилище этих самых файлов.



Доступ к файлам в Python

Чаще всего разработчики в своей работе сталкиваются с обработкой данных, которые хранятся в файлах. А файлы, в свою очередь, обычно хранятся на устройствах хранения данных — жестких, оптических, сетевых или твердотельных накопителях.

Легко представить программу, которая сортирует 20 чисел, и столь же легко представить пользователя этой программы, который вводит эти 20 чисел прямо с клавиатуры.

Гораздо сложнее представить ту же самую задачу, когда нужно отсортировать 20 000 чисел. Ну а пользователей, которые могли бы ввести все эти числа без ошибок, просто не существует.

Намного проще представить, что эти цифры хранятся в файле на диске, который считывает программа. Программа сортирует числа и не отправляет их на экран, а создает новый файл и сохраняет там отсортированную последовательность чисел.

Если нужно реализовать простую базу данных, единственный способ сохранить информацию между запусками программы — сохранить ее в файл (или файлы, если база данных более сложная).



Доступ к файлам в Python

В принципе, любая сложная задача в программировании зависит от использования файлов, независимо от того, обрабатываются ли изображения (которые хранятся в файлах), умножаются ли матрицы (которые хранятся в файлах) или рассчитываются зарплата и налоги (путем считывания данных, хранящихся в файлах).

Может возникнуть вопрос, почему мы так долго ждали, чтобы поговорить об этих проблемах.

Ответ очень прост — способ доступа и обработки файлов в Python реализован с использованием согласованного набора объектов. И сейчас, на мой взгляд, идеальный момент, чтобы поговорить об этом.





Имена файлов

Разные операционные системы обрабатывают файлы по-разному. Например, в операционных системах Windows и Unix/Linux используются разные соглашения об именах.

Если мы напишем каноническое имя файла (имя, которое однозначно определяет местоположение файла, независимо от его уровня в дереве каталогов), то увидим, что в Windows и в Unix/Linux эти имена выглядят по-разному



Как видите, Unix-подобные системы не используют букву диска (например, «C:») и все каталоги растут из одного корневого каталога с именем «/», а Windows распознает корневой каталог как «\».

Кроме того, имена системных файлов в Unix/Linux чувствительны к регистру. Windows сохраняет регистр букв в имени файла, но сам регистр для него не имеет значения. То есть эти две строки:

- ThisIsTheNameOfTheFile
- thisisthenameofthefile

описывают два разных файла в Unix/Linux, но Windows воспринимает их как одно и то же имя одного файла.



Имена файлов

Главное и самое яркое отличие в том, что приходится использовать два разных разделителя для имен каталогов: «\» в Windows и «/» в Unix/Linux.

Эта разница не очень важна для обычного пользователя, но **очень важна при написании программ на Python.**

Чтобы понять, почему это так, попробуйте вспомнить очень специфическую роль, которую играет \ внутри строк Python.

Предположим, вам нужен конкретный файл, расположенный в каталоге dir, и этот файл называется «file». Также предположим, что вы хотите задать строку, содержащую имя файла. В Unix-подобных системах это может выглядеть следующим образом:

```
name = "/dir/file"
```

Но если попытаться задать строку для Windows следующим образом:

```
name = "\dir\file"
```

то получится неприятный сюрприз: либо Python выдаст ошибку, либо программа выполнится так, будто имя файла как-то искажено.

На самом деле, это вполне очевидный и естественный результат. Python использует символ «\» для экранирования символов (\n).

Имена файлов

Это означает, что имена файлов в Windows должны быть записаны следующим образом:

name = "\\dir\\file"

К счастью, есть еще одно решение. Python умеет преобразовывать косые черты в обратную косую черту каждый раз, когда обнаруживает, что этого требует ОС. Это означает, что следующие назначения:

name = "/dir/file"

name = "c:/dir/file"

будут работать и в Windows.

Любая программа, написанная на Python (и не только на Python, поскольку это соглашение применяется практически ко всем языкам программирования), взаимодействует с файлами не напрямую, а через некоторые абстрактные объекты, которые по-разному называются в разных языках или средах. Наиболее часто используемые термины это дескрипторы (handles) или потоки данных (streams) (мы будем использовать их как синонимы).

Если в арсенале программиста имеется более или менее богатый набор функций/методов, то он может использовать механизмы ядра операционной системы для выполнения определенных операций с потоками данных, которые влияют на реальные файлы.



Имена файлов

Таким образом, можно организовать доступ к любому файлу, даже если имя файла неизвестно на момент написания программы. Операции, выполняемые с абстрактным потоком данных, отражают действия, связанные с физическим файлом.

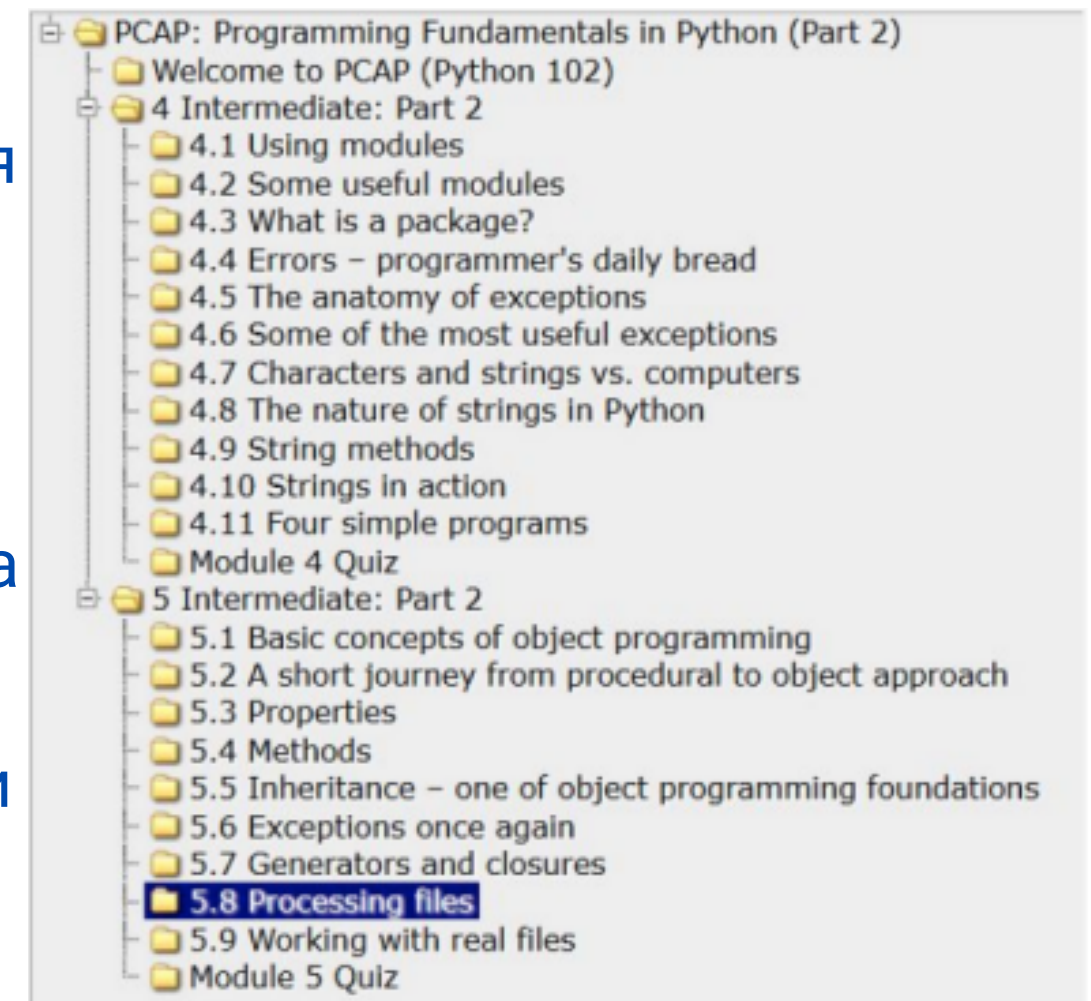
Чтобы связать поток данных с файлом, необходимо выполнить явную операцию/

Операция связывания потока данных с файлом называется открытием файла, а прерывание этой связи называется закрытием файла.

Отсюда можно сделать вывод, что самая первая операция, выполняемая в потоке данных, — всегда `open`, а последняя — `close`. По сути, программа может свободно управлять потоком данных между этими двумя событиями и обрабатывать связанный файл.

Эта свобода, конечно, ограничена физическими характеристиками файла и способом, которым файл был открыт.

Стоит повторить, что у вас может не получиться открыть поток данных, и это может произойти по нескольким причинам. Наиболее распространенной является отсутствие файла с указанным именем.



Имена файлов

А еще бывает так, что физический файл существует, но программе запрещено его открывать. Или, например, программа открыла слишком много потоков данных, а конкретная операционная система не разрешает одновременно открывать более n количества файлов (например, более 200).

Хорошо написанная программа должна обнаруживать эти сбои и реагировать соответствующим образом.

Файлы в операционной системе





Пути к файлам в операционной системе (ОС)

Работу с компьютером сложно представить без операционной системы, с которой работает каждая программа. Поэтому для удобной коммуникации с ней в Python есть стандартная библиотека `os`.

Известно, что для получения информации необходимо пройти определенный маршрут, который принято называть `path` (путь). Он, в свою очередь, может быть абсолютным или относительным.

Абсолютный – тот, что начинается с корневых директорий системы, примером которых могут быть системные диски операционной системы Windows.

Относительный путь – тот, что прокладывается от места, в котором располагается программа или пользователь.





Как попасть к файлу функция walk()

Пройтись по указанному пути можно с помощью функции walk(), куда он передается в качестве параметра. Запишем:

```
import os

path_learning = r'D:\work_Top_Academy_children\test dir for module 10'

for path, dirnames, filenames in os.walk(path_learning):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
        v   v   v
path - D:\work_Top_Academy_children\test dir for module 10
dirnames - ['test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
path - D:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
```

Как видим, пройдясь по указанному пути функцией walk(), мы поочередно получили пути и названия всех файлов и директорий, содержащихся в нем.

Также обратите внимание на литерал "r" перед строкой в которой указан путь, он делает строку "сырой", это значит что все символы в этой строке будут читаться как есть (то есть управляющие и другие символы которые могут нести некий функционал становятся просто символами).



Модуль `os.path`.

Однако в разных ОС путь указывается по-разному, как быть в этом случае?

Корректную работу с ними обеспечит модуль **`os.path`**. А для того, чтобы путь указывался корректно независимо от системы, необходимо передать его в качестве аргумента в функцию **`normpath()`**. Например, в ОС Linux в качестве знака препинания пути используется `/`. Заменим `\` на `/` в нашем коде, имитируя путь с Linux и передадим его вышеуказанной функции:

```
import os

path = os.path.normpath(r"D:/work_Top_Academy_children/test dir for module 10")
for path, dirnames, filenames in os.walk(path):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
        v   v   v

path - D:\work_Top_Academy_children\test dir for module 10
dirnames - ['test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
path - D:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
```

Функция `normpath()` модуля `os.path` нормализует имя пути, свернув избыточные разделители и ссылки верхнего уровня, чтобы `A//B`, `A/B/`, `A./B` и `A/foo/../B` все стали `A/B`.

Эта манипуляция строк может изменить значение пути, который содержит символические ссылки. В Windows он преобразует прямые косые черты в обратные.

Аргумент `path` может принимать байтовые или текстовые строки. Результатом будет являться передаваемый тип.



Модуль `os.path.join()`

Как было видно из предыдущих примеров, путь указывается строкой, но часто возникает вопрос его модификации. Конечно, можно добавить строки и получить новый путь, но лучше воспользоваться функцией `join()`, которая не только добавит их, а и установит корректный знак препинания, в соответствии с операционной системой:

```
import os

disk = "d:\\"
dir_1 = "work_Top_Academy_children"
dir_2 = "test dir for module 10"
path = os.path.join(disk, dir_1, dir_2)
print(path)
for path, dirnames, filenames in os.walk(path):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
```

>>> d:\work_Top_Academy_children\test dir for module 10
path - d:\work_Top_Academy_children\test dir for module 10
dirnames - ['test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
path - d:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']

Как видим в аргументы функции `.join()`, переданы 3 аргумента, диск на котором содержится наша директория "d:\\", директория "work_Top_Academy_children" - на этом диске и вложенная в нее директория "test dir for module 10"



Модуль `os.path.isabs()/isfile()/isdir()/islink()`

Кроме того, с помощью функции **isabs()** можно проверить, является ли путь абсолютным. А с помощью функций **isfile()**, **isdir()**, **islink()** узнаем, ведет путь к файлу, директории или ссылке:

```
import os

path_dir = os.path.normpath(r"D:\work_Top_Academy_children\test dir for module 10")
print(os.path.isabs(path_dir))
print(os.path.isfile(path_dir))
print(os.path.isdir(path_dir))
print(os.path.islink(path_dir))
```

>>> True
False
True
False

- данный путь является абсолютным (True);
- он не ведет к файлу (False);
- он ведет к директории (True);
- он не ведет к ссылке (False).

```
import os

path_file = os.path.normpath(r"D:\work_Top_Academy_children\test dir for module 10\test_file_1.DOCX")
print(os.path.isabs(path_file))
print(os.path.isfile(path_file))
print(os.path.isdir(path_file))
print(os.path.islink(path_file))
```

>>> True
True
False
False

- данный путь является абсолютным (True);
- он ведет к файлу (True);
- он не ведет к директории (False);
- он не ведет к ссылке (False).



Модуль os.path.mkdir()/rmdir

За создание директорий в os отвечает функция mkdir(), куда передается путь, последней частью которого и выступает название новой директории:

```
import os

base_path = r"D:\work_Top_Academy_children\test dir for module 10"
for path, dirnames, filenames in os.walk(base_path):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")

path_dir = os.path.normpath(r"D:\work_Top_Academy_children\test dir for module 10\my_new_dir")
os.mkdir(path_dir)

for path, dirnames, filenames in os.walk(base_path):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
```

```
path - D:\work_Top_Academy_children\test dir for module 10
dirnames - ['test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX', 'test_file_2_link.lnk']
>>> path - D:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
path - D:\work_Top_Academy_children\test dir for module 10

dirnames - ['my_new_dir', 'test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX', 'test_file_2_link.lnk']
path - D:\work_Top_Academy_children\test dir for module 10\my_new_dir
dirnames - []
filenames - []
path - D:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
```

Наша программа выводит нам информацию до функции mkdir() и после нее, в примере выделены строки с именами директорий до и после того как функция отработала, и как видим там появилась новая папка.



Модуль os.path.mkdir()/rmdir

А удалить директории позволит функция rmdir(), в которую так же передается путь:

```
import os

base_path = r"D:\work_Top_Academy_children\test dir for module 10"
for path, dirnames, filenames in os.walk(base_path):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")

path_dir = os.path.normpath(r"D:\work_Top_Academy_children\test dir for module 10\my_new_dir")
os.rmdir(path_dir)
for path, dirnames, filenames in os.walk(base_path):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
```

>>> dirnames - ['my_new_dir', 'test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX', 'test_file_2_link.lnk']
path - D:\work_Top_Academy_children\test dir for module 10\my_new_dir
dirnames - []
filenames - []
path - D:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']

>>> path - D:\work_Top_Academy_children\test dir for module 10
dirnames - ['test_dir_1']
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX', 'test_file_2_link.lnk']
path - D:\work_Top_Academy_children\test dir for module 10\test_dir_1
dirnames - []
filenames - ['test_file_1.DOCX', 'test_file_2.DOCX']
path - D:\work_Top_Academy_children\test dir for module 10

Наша программа выводит нам информацию до функции rmdir() и после нее, в примере выделены строки с именами директорий до и после того как функция отработала, и как видим было две папки а стала одна.

Файловые потоки



Файловые потоки

Открытие потока данных не только связано с файлом, но и должно объявлять способ обработки потока. Такое объявление называется **режимом открытия файла**.

Если открытие прошло успешно, **программа получит разрешение на выполнение только тех операций, которые соответствуют заявленному режиму открытия**.

В потоке данных выполняются две основные операции:

- **чтение из потока:** части данных извлекаются из файла и помещаются в область памяти, которая управляется программой (например, в переменную);
- **запись в поток:** часть данных памяти (например, переменная) передается в файл.

Для открытия потока используются три основных режима:

- **режим чтения:** поток, открытый в этом режиме, **разрешает только операции чтения**; попытка записи в поток вызовет исключение (которое называется `UnsupportedOperation`, оно наследует `OSError` и `ValueError` и является потомком модуля `io`);
- **режим записи:** поток, открытый в этом режиме, **разрешает только операции записи**; попытка прочитать поток вызовет исключение, упомянутое выше;
- **режим обновления:** поток, открытый в этом режиме, **разрешает и запись, и чтение**.

Файловые потоки

Прежде чем мы обсудим, как управлять потоками, мы должны кое-что объяснить. *Поведение потока очень похоже на принцип работы магнитофона.*

Когда вы что-то читаете из потока, виртуальная головка перемещается по потоку в соответствии с количеством байтов, переданных из потока.

Когда вы записываете что-то в поток, одна и та же головка движется вдоль потока, записывая данные из памяти.

Всякий раз, когда мы говорим о чтении и записи в поток, попытайтесь представить эту аналогию. В книгах по программированию этот механизм называется **текущая позиция файла**, и мы тоже будем использовать этот термин.



Файловые дескрипторы



Файловые дескрипторы

Python предполагает, что **каждый файл спрятан за объектом соответствующего класса**.

Сразу же возникает вопрос, как интерпретировать слово «соответствующий».

Файлы могут быть обработаны разными способами. Некоторые из них зависят от содержимого файла, другие — от целей программиста.

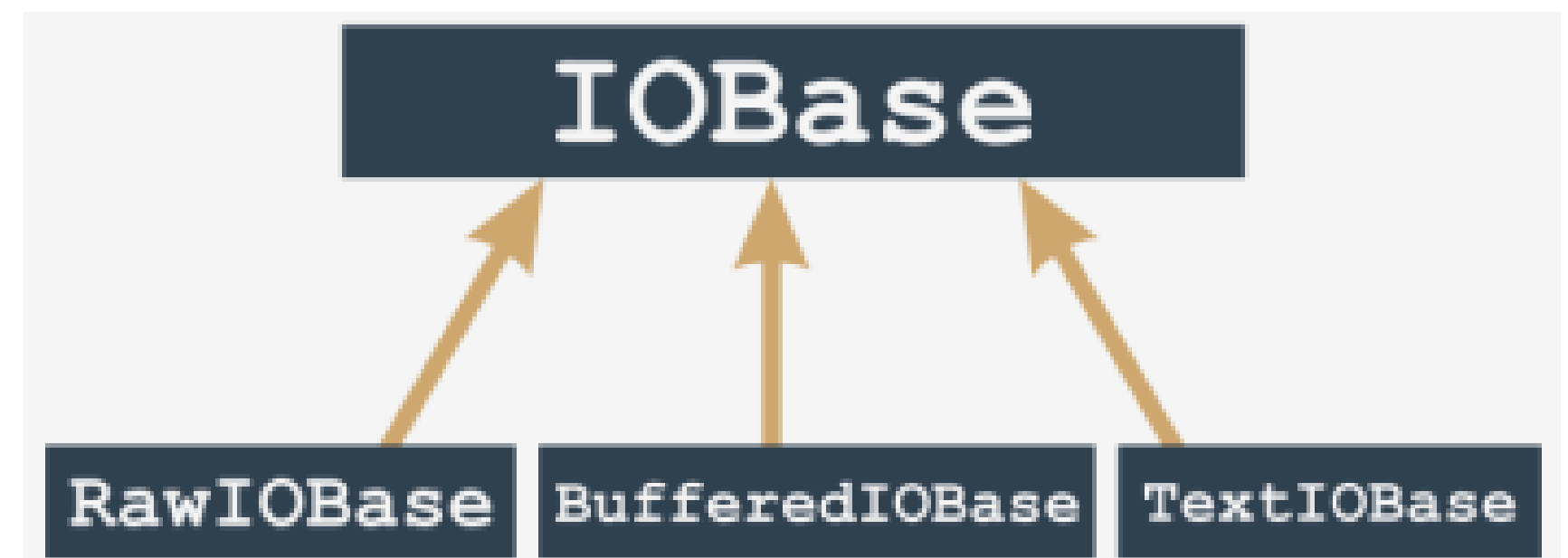
В любом случае, разные файлы могут требовать разных наборов операций и вести себя по-разному.

Объект соответствующего класса создается при открытии файла и уничтожается при закрытии.

Между этими двумя событиями объект указывает, какие операции должны выполняться в конкретном потоке. Разрешенные операции зависят от способа открытия файла.

В целом, этот объект происходит от одного из классов, показанных справа:

Примечание: *конструкторы никогда не используются для вызова этих объектов. Единственный способ их получить — это вызвать функцию `open()`.*



Файловые дескрипторы

Функция анализирует предоставленные аргументы и автоматически создает требуемый объект.

Чтобы **закрыть объект, нужно вызвать метод `close()`**.

Вызов разорвет соединение с объектом и файлом и удалит объект.

Пока что мы будем заниматься только потоками, которые представлены объектами **BufferIOBase** и **TextIOBase**. Скоро вы поймете, почему.

Из-за типа содержимого потока, все потоки делятся на текстовые и двоичные.

- Текстовые потоки структурированы в строки; то есть они содержат типографские символы (буквы, цифры, знаки пунктуации и т.д.), расположенные в строках. Это видно невооруженным глазом при просмотре содержимого файла в редакторе. Чаще всего этот файл записывается (или читается) символ за символом или строка за строкой.
- Двоичные потоки содержат не текст, а последовательность байтов любого значения. Эта последовательность может быть, например, исполняемой программой, изображением, аудио- или видео-файлом, файлом базы данных и т.д. Поскольку эти файлы не содержат строк, операции чтения и записи устанавливают связи с данными любого размера. Следовательно, данные читаются/записываются байт за байтом или блок за блоком. Размер блока обычно колеблется от одного до произвольно выбранного значения

Файловые дескрипторы

Отсюда вытекает неочевидная проблема. В Unix/Linux системах концы строки отмечены одним символом LF (код ASCII: 10), который обозначается в Python как `\n`.

Другие операционные системы, особенно наследницы древней операционной системы CP/M (это относится и к системам семейства Windows), используют другое соглашение: конец строки отмечен парой символов: **CR** и **LF** (ASCII коды: 13 и 10), которые можно кодировать как `\r\n`.

Эта двусмысленность может вызвать разные неприятные последствия.

Если вы создаете программу обработки текстового файла, которая написана для Windows, то концы строк распознаются по символам `\r\n`, но та же самая программа будет совершенно бесполезна в среде Unix/Linux, и наоборот — программа, написанная для систем Unix/Linux, может быть бесполезна в Windows.

О программах, которые нельзя использовать в разных средах, говорят, что они «не поддаются портированию (non-portable)».





Файловые дескрипторы

Особенность программы, которая позволяет ей выполняться в разных средах, называется «портируемостью (portability)». А сама программа, наделенная такой особенностью, называется «переносимой (портируемой) программой (portable program)».

Поскольку проблема переносимости всегда стояла остро, было решено организовать все таким образом, чтобы избавить разработчика от необходимости вникать в эти тонкости.

Саму проблему решили на уровне классов, которые отвечают за чтение и запись символов в потоке. Вот как это работает:

- когда поток открыт и есть рекомендация обработать данные в связанном файле как текст (или такой рекомендации нет), то он **переключится в текстовый режим**;
- во время чтения/записи строк в среде Unix ничего особенного не происходит, но когда те же операции выполняются в среде Windows, происходит процесс, который называется преобразованием символов новой строки: когда вы читаете строку из файла, каждая пара символов `\r\n` заменяется одним символом `\n` и наоборот. Во время операций записи каждый символ `\n` заменяется парой символов `\r\n`;
- этот механизм абсолютно **понятен** программе, поэтому программист может писать ее так, как если бы она предназначалась только для обработки текстовых файлов в Unix/Linux, потому что этот исходный код и в среде Windows будет работать правильно;
- когда поток открыт, и есть соответствующая рекомендация, его содержимое принимается как есть, **без преобразований** — ни один байт не добавляется и не удаляется.

Открытие потоков



Открытие потока

Открытие потока выполняет функция, которую можно вызвать следующим образом:

```
stream = open(file, mode = 'r', encoding = None)
```

Давайте проанализируем

- имя функции (open) говорит само за себя; если открытие прошло успешно, функция возвращает объект потока; в противном случае возникает исключение (например, FileNotFoundError если файла, который вы собираетесь прочесть, не существует);
- первый параметр функции (file) задает имя файла, который будет связан с потоком;
- второй параметр (mode) задает режим открытия для этого потока; это строка, заполненная последовательностью символов, и каждый из них имеет свое особое значение (скоро разберем подробнее);
- третий параметр (encoding) указывает тип кодировки (например, UTF-8 при работе с текстовыми файлами).
- открытие должно быть самой первой операцией, выполняемой в потоке.

Примечание: аргументы режима и кодирования могут быть опущены, тогда принимаются значения по умолчанию. Режим открытия по умолчанию — это чтение в текстовом режиме, а кодировка по умолчанию зависит от используемой платформы.



Открытие потоков: режимы

r: чтение

- поток будет открыт в режиме **чтения**;
- файл, связанный с потоком **должен существовать** и быть доступным для чтения, в противном случае, функция `open()` вызывает исключение.

w: запись

- поток будет открыт в режиме **записи**;
- файл, связанный с потоком, **не обязан существовать**. Если его не существует, он будет создан. Если существует, он будет обрезан до нулевой длины (стерт). Если создание невозможно (например, из-за системных разрешений) функция `open()` вызывает исключение.

a: дозапись

- поток будет открыт в режиме **дозаписи**;
- файл, связанный с потоком, **не обязан существовать**. Если его не существует, он будет создан. Если существует, виртуальная записывающая головка будет установлена в конец файла (предыдущее содержимое файла останется без изменений).



Открытие потоков: режимы

r+: чтение и обновление

- поток будет открыт в режиме **чтения и обновления**;
- файл, связанный с потоком **должен существовать** и быть доступным как для чтения так и для записи, в противном случае, функция `open()` вызывает исключение.
- операции чтения и записи разрешены для потока

w+: запись и обновление

- поток будет открыт в режиме **записи**;
- файл, связанный с потоком, **не обязан существовать**. Если его не существует, он будет создан. Предыдущее содержимое файла остается нетронутым;
- операции чтения и записи разрешены для потока



Выбор текстового и бинарного режимов

Если в конце строки режима стоит буква «b», это означает, что поток должен быть открыт в двоичном режиме (binary mode).

Если в конце строки режима стоит буква «t», поток должен быть открыт в текстовом режиме (text mode).

Текстовый режим — поведение, которое подразумевается по умолчанию, если в строке нет спецификатора двоичного/текстового режима.

Наконец, успешное открытие файла установит текущую позицию файла (виртуальную головку чтения/записи) перед первым байтом файла, если выбранный режим — не a, и после последнего байта файла, если выбранный режим — a.

Текстовый режим	Двоичный режим	Описание
rt	rb	чтение
wt	wb	запись
at	ab	дозапись
r+t	r+b	чтение и обновление
w+t	w+b	чтение и обновление

Файл также можно открыть для эксклюзивного создания. За это отвечает режим открытия x. Если файл уже существует, функция `open()` вызовет исключение.

Чтение из файла





Работа с файлами как открыть и прочитать файл

Для работы с файлами нам прежде всего нужно этот файл создать:

1) Для этого в PyCharm выбираем команду создать файл и создаем файл с нужным нам именем, например: learning.txt (указать формат файла нужно обязательно), и вносим туда данные.

learning.txt	×
1	hello
2	world

Для работы с этим файлом в нашей программе создаем переменную file и присваиваем сюда вызов функции **open()**

```
1 file = open("learning.txt", "rt", encoding="utf-8")
2 content = file.read()
3 file.close()
print(content)
```

>>> hello
world

в функцию передаются следующие параметры:

1) **open**("путь к файлу", действие с ним в данном случае - "read text", "utf-8" - кодировка);

Далее

2) **read()** - чтения из файла, в скобках указываются байты (по умолчанию читает все);

3) **file.close()** - закрыть файл, чтобы он не загружал систему.

Однако это не очень рационально т.к. надо каждый раз вручную закрывать наш файл, что можно и забыть, поэтому для того чтобы не закрывать вручную файл есть другая конструкция:

```
with open("learning.txt", "rt", encoding="utf-8") as file:
    content = file.read()
print(content)
```

>>> hello
world

Данная конструкция означает что мы будем открывать некий ресурс как файл, а в ее теле совершать с ним необходимые нам действия.

with это ключевое слово которое позволяет открыть файл как некий ресурс и закрыть его автоматически когда мы закончим с ним работать (работает для чтения, перезаписи, добавления информации, создания)



Работа с файлами как открыть и прочитать файл

Информацию из файла можно считать также и циклом:

```
file = open("learning.txt", "rt", encoding="utf-8")
for line in file:
    print(line)
file.close()
```

>>> hello
>>> world

```
with open("learning.txt", "rt", encoding="utf-8") as file:
    for line in file:
        print(line)
```

>>> hello
>>> world

```
file = open("learning.txt", "rt", encoding="utf-8")
for line in file:
    print(line, end="")
file.close()
```

>>> hello
>>> world

```
with open("learning.txt", "rt", encoding="utf-8") as file:
    for line in file:
        print(line, end="")
```

>>> hello
>>> world

Как видно из примеров, мы получили похожий результат, однако, у нас есть строка пропуска, чтобы этого избежать можно модифицировать код следующим образом:

Используя параметры форматирования функции, print() а именно end="", мы убрали не нужный нам разрыв, либо используя .rstrip(), но это в следующем примере

То каким способом читать файл, мы выбираем в зависимости от того что в этом файле находится.



Работа с файлами как открыть и прочитать файл пример:

Допустим у нас есть список гостей guests.txt, нам необходимо его считать и вывести их количество:

1й способ:

```
guests_count = 0
with open('guests.txt', 'r', encoding="UTF-8") as file:
    for line in file:
        print(line.rstrip())
        guests_count += 1
print(f"Количество гостей: {guests_count}")
```

>>> Тодд Говард
Джон Кармак
Том Холл
Адриан Кармак
Джон Ромеро
Лиза Су
Количество гостей: 6

2й способ:

```
with open('guests.txt', 'r', encoding="UTF-8") as file:
    guests = file.readlines()
    print(guests) # добавлено для информации
    guests_count = len(guests)

print(''.join(guests))
print(f"Количество гостей: {guests_count}")
```

>>> ['Тодд Говард\n', 'Джон Кармак\n', 'Том Холл\n', 'Адриан Кармак\n', 'Джон Ромеро\n', 'Лиза Су']
Тодд Говард
Джон Кармак
Том Холл
Адриан Кармак
Джон Ромеро
Лиза Су
Количество гостей: 6

В данном способе мы применили метод `readlines()`, который считывает файл построчно и содержимое строк помещает в список (даже символ переноса), есть также метод `readline()` который читает одну строку.

Запись в файл





Работа с файлами как записать информацию в файл

Файлы можно не только открывать но и при помощи кода создавать файлы, записывать туда информацию, а также добавлять, что-то уже к существующей информации. Рассмотрим на примере:

```
with open('write_data.txt', 'wt', encoding='utf-8') as file:
```

```
    file.write("Hello\n") # write для последовательности символов writelines для последовательности строк
```

```
    file.write("World\n")
```

```
    file.write("Newline_text")
```

>>>

write_data.txt	
1	Hello
2	World
3	Newline_text

Запись в файл происходит при помощи параметра "w", а в данном случае "wt" (write text) - если файл не существует он будет создан под заданным в параметрах именем. Также важно знать что **параметр "w" именно что перезаписывает файл а не добавляет к нему новые данные.** Для того чтобы добавить что-то к уже существующей информации используется параметр "a" сокращенно от append.

```
with open('write_data.txt', 'at', encoding='utf-8') as file:
```

```
    file.write("Hello\n") # write для последовательности символов writelines для последовательности строк
```

```
    file.write("World\n")
```

```
    file.write("Newline_text\n\n")
```

>>>

write_data.txt	
1	Hello
2	World
3	Newline_text
4	
5	Hello
6	World
7	Newline_text

В данном примере я 2 раза запустил данный код и в результате в файле мы получили 2 раза записанный текст (кстати параметр "a" также умеет создавать файлы если они отсутствуют)



Работа с файлами как записать информацию в файл пример:

Например мы задаем пользователю какие-то вопросы и хотим записать его ответы в файл

```
user_language = input("Какой язык вы учите? ")
user_time = input("Как давно? ")
user_institution = input("Где? ")

user_answers = [user_language, user_time, user_institution]

with open('user_answers.txt', 'w', encoding='utf-8') as file:
    for answer in user_answers:
        file.write(f"{answer}\n")

print("Ответы записаны!")
```

>>>

>>>

Работа программы

Какой язык вы учите? *Python*

Как давно? *5 лет*

Где? *Везде*

Ответы записаны!

Результат записи

≡ user_answers.txt ×

1	Python
2	5 лет
3	Везде
4	

Обратите внимание полученные ранее знания по циклам можно применять и при записи информации в файл, также нужно обратить внимание на то что в данном случае мы используем просто "w", это возможно потому что python все по умолчанию записывает как текст поэтому дополнительную инструкцию t к параметру, можно не указывать (это относится и к параметрам "r" и "a")

Свой формат





Что еще можно хранить в файлах

В файлах могут храниться не только строки, в файлах можно хранить списки, словари, несколько словарей и прочие типы данных то мы можем использовать разделители, например разделить их точкой с запятой или двумя вертикальными черточками, а на самом деле чем угодно.

При прочтении текста из файла мы можем превратить его в словарь при помощи оператора `.split()`.

Для понимания сразу же посмотрим это на практике:

У нас есть файл в котором через двоеточие указаны имя и фамилия: к какой компании имеет отношение

<pre>own_data.txt x 1 Тодд Говард: Bethesda 2 Джон Кармак: ID Software 3 Лиза Су: AMD</pre>	нашей задачей будет преобразовать эти данные в строку вида:	Тодд Говард работает в Bethesda Джон Кармак работает в ID Software Лиза Су работает в AMD
<pre>with open("managers.txt", encoding="utf-8") as managers_data: for manager_data in managers_data: data = manager_data.rstrip('\n').split(":") manager = data[0] company = data[1] # manager, company = manager_data.rstrip('\n').split(":") print(f"{manager} работает в {company}!")</pre>	>>>	Тодд Говард работает в Bethesda! Джон Кармак работает в ID Software! Лиза Су работает в AMD!



Что еще можно хранить в файлах пример

В данном примере нашей задачей будет написать программу которая будет вести учет покупок и расходов исходя из нашего файла, при запуске программа должна вывести количество видов товаров и общую сумму по ним:

```
row_index = 0
1 items_count = 0
  items_sum = 0

2 with open('shopping.txt', encoding='utf-8') as file:
    3 for item_data in file:
        4 row_index += 1
        5 if item_data.count(':') < 2:
            print(f"В строке {row_index} есть ошибка")
            continue

        6 temp_data = item_data.strip().split(":")
        print(temp_data) # показывает какие данные мы обрабатываем
        7 item, quantity, price = temp_data
        8 items_count += 1
        items_sum += float(price) * float(quantity)

9 print(f"В списке {items_count} позиций. Сумма {items_sum} рублей")
```

>>>

```
['Молоко пак', '2', '200']
['Масло пач', '1', '300']
['Сыр кг', '0.5', '400']
['Помидоры кг', '1', '600']
['Огурцы кг', '2', '350']
['Перец кг', '0.8', '900']
В списке 6 позиций. Сумма 2920.0 рублей
```

- 1) Задаем базовые переменные в которые мы записываем наши данные
- 2) Открываем файл
- 3) Циклом проходимся по строкам в файле
- 4) Считаем ряды
- 5) Условие проверки корректности данных
- 6) Превращаем данные в список по разделителю
- 7) Сохраняем данные в переменные
- 8) Производим расчеты
- 9) Выдаем результат